API Spec

The contract between the Next.js frontend, the FastAPI orchestrator, and any MCP client. If it is not in this spec, it does not ship in the MVP.

Version: API v1 - spec 1.0, 17 October 2026
Base URL: `https://api.aivideostudio.app/v1`
Stack: Next.js frontend, FastAPI orchestrator, Redis + Celery workers, faster-whisper, Remotion, ffmpeg, R2/S3 storage
Companion docs: PRD, Architecture and Data, AI Pipeline, Security and Licensing

Conventions that apply to every endpoint

- **Format:** JSON over HTTPS, UTF-8. Media never passes through the API - it moves by signed URL (Section 9).
- **IDs:** Prefixed ULIDs: `prj_`, `job_`, `ast_`, `usr_`, `key_`, `whk_`. IDs are opaque; clients never parse them.
- **Time:** ISO 8601 UTC everywhere (`2026-10-17T04:20:00Z`). Durations in seconds as numbers, never strings.
- **Pagination:** Cursor-based. Request: `?cursor=&limit=` (default 25, max 100). Response adds `next_cursor`, null at the end. No offset pagination.
- **Long work is always a job.** Any operation over about 2 seconds returns `202 Accepted` with a `job_id`. The API never holds a request open for research, generation, or render.
- **Stage state machine:** Every project carries `stage_status` for each of the 10 stages: `not_started | queued | running | needs_review | done | failed`. The frontend renders progress from this object alone - no extra status calls.

> **Clean-room note:** Endpoint shapes here are original to this product. Patterns borrowed from comparable editors - proposal-based editing, timeline-as-data - are implemented from scratch. No code, text, or schema is copied from Slot Studio, OpenChatCut, Palmier Pro, or Pallaidium.

1. Authentication and authorization

Two credential types. The browser uses a session token; automation and MCP clients use an API key. Both resolve to the same user and workspace, and every rule below applies equally.

Credential	How it is sent	Rules
Session JWT	`Authorization: Bearer <jwt>`	Issued by `POST /auth/login` and `POST /auth/signup`. Access token 60 minutes, refresh token 30 days, rotated on use. Stored httpOnly by the Next.js layer; the browser never sees the refresh token in JavaScript.
API key	`Authorization: Bearer avs_live_...`	Created in settings, shown once, stored hashed (argon2id). Scoped: `projects:read`, `projects:write`, `jobs:read`, `billing:read`. Revocable instantly. Max 5 active keys per workspace in MVP.
Signed media URL	Query-signed R2/S3 URL	Issued per asset, 15-minute expiry for reads, 60 minutes for uploads. Never a credential on its own - it grants access to one object only.

Roles inside a workspace

Role	Can do	Cannot do
`owner`	Everything, including billing, keys, member removal	-
`editor`	Create and edit projects, run stages, render, export publish packs	Billing, API keys, member management
`viewer`	Read projects, watch previews, download finished exports	Edit, run stages, render, spend credits

Credit-spending endpoints (generation, render, AI voice) check role *and* balance before queuing. A viewer calling them gets `403 FORBIDDEN_ROLE`, never a queued job.

Versioning

- Path-versioned: `/v1`. Breaking changes go to `/v2`; v1 stays live for at least 6 months after v2 launches, with a `Sunset` header announced 90 days ahead.
- Non-breaking additions (new optional fields, new endpoints) ship inside v1 without notice. Clients must ignore unknown fields.
- Every response carries `X-API-Version: 1` and a `request_id` (`req_...`) that support and logs use to trace the call.

2. Errors, rate limits, idempotency

One error envelope for everything

```
{ "error": { "code": "INSUFFICIENT_CREDITS",
             "message": "Render needs 42 credits, balance is 11.",
             "request_id": "req_01K...", "details": { "needed": 42, "balance": 11 } },
  "data": null }
```

Success responses mirror it: `{ "data": { ... }, "error": null }`. Clients branch on `code`, not on message text.

HTTP	Code	Meaning and client action
400	`VALIDATION_ERROR`	Body failed schema. `details.fields` lists each bad field. Fix and retry.
401	`UNAUTHENTICATED`	Missing, expired, or revoked credential. Refresh once, then send the user to login.
403	`FORBIDDEN_ROLE` / `FORBIDDEN_SCOPE`	Role or key scope does not allow this call. Do not retry.
404	`NOT_FOUND`	Does not exist or is not in this workspace - same response either way, no existence leaks.
409	`STAGE_LOCKED` / `VERSION_CONFLICT`	Stage is running, or timeline version is stale. Re-read, then retry.
402	`INSUFFICIENT_CREDITS`	Balance below the job estimate. Nothing was queued or charged.
422	`STAGE_PREREQUISITE_MISSING`	An earlier stage is not `done` / `needs_review` - e.g. render before a timeline exists.
429	`RATE_LIMITED`	Slow down. Honour `Retry-After` (seconds). See limits below.
500 / 503	`INTERNAL` / `WORKER_UNAVAILABLE`	Our fault. Safe to retry with backoff; the `request_id` goes in any support report.

Rate limits (per workspace, sliding window)

Bucket	Free trial	Creator	Studio
Reads (GET)	300 / min	600 / min	1,500 / min
Writes (POST/PATCH/DELETE)	60 / min	120 / min	300 / min
Job submissions	10 / min, 2 concurrent renders	20 / min, 4 concurrent	40 / min, 8 concurrent
Auth attempts	10 / 15 min per IP and per account - plan-independent		

Responses carry `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` on every call, so client backoff can be coded against published limits.

Idempotency

Every POST that creates a resource, starts a job, or moves credits requires an `Idempotency-Key` header (client-generated UUID). Replaying the same key within 24 hours returns the original response - same `job_id`, no second charge. A key reused with a different body returns `409 IDEMPOTENCY_KEY_REUSED`.

3. Projects and the stage pipeline

A project is the aggregate root. Everything - script, voice, timeline, renders - hangs off one project ID, and the 10 user-facing stages map to the sub-resources below in order.

#	Stage	Sub-resource	Run endpoint
1	Topic	`brief` on the project	`POST /projects`
2	Research	`/research`	`POST /projects/{id}/research/run`
3	Script	`/script`	`POST /projects/{id}/script/run`
4-5	Voice + cleanup	`/voice`	`POST /projects/{id}/voice/run`, `.../voice/cleanup`
6	Visual plan	`/visual-plan`	`POST /projects/{id}/visual-plan/run`
7	Generation into project	`/assets`	`POST /projects/{id}/assets/generate`
8	Motion graphics	`/motion`	`POST /projects/{id}/motion/run`
9	Timeline edit	`/timeline`	`PATCH /projects/{id}/timeline` (not a job)
10a	Captions + sound	`/captions`, `/audio-mix`	`POST .../captions/run`, `POST .../audio-mix/run`
10b	Preview, render, publish	`/renders`, `/publish-pack`	`POST /projects/{id}/renders`, `POST .../publish-pack/run`

Core project endpoints

Endpoint	Notes
POST `/projects`	Create from a brief. Body: `title`, `topic`, `target_length_sec` (60-1800), `aspect` (`16:9` MVP; `9:16` derived in publish pack), `language` (BCP-47). Returns `201` with the full project.
GET `/projects`	List, newest first. Filters: `status`, `q` (title search). Cursor pagination.
GET `/projects/{id}`	Full project including `stage_status`, `credit_estimate` for remaining stages, and links to current sub-resources.
PATCH `/projects/{id}`	Brief fields only: title, topic, target length, language. Editing the topic after research is `done` marks research and script `stale: true` - it never silently deletes their output.
DELETE `/projects/{id}`	Soft delete. Project and assets are restorable for 30 days via `POST /projects/{id}/restore`, then purged.
POST `/projects/{id}/duplicate`	Copies brief, script, and timeline. Generated assets are referenced, not re-billed.

```
{ "id": "prj_01K8...", "title": "Why rent keeps rising", "target_length_sec": 420,
  "aspect": "16:9", "language": "en-IN", "status": "in_progress",
  "stage_status": { "research": "done", "script": "needs_review", "voice": "not_started" },
  "credits_spent": 18, "created_at": "2026-10-17T04:20:00Z" }
```

4. Creation stages: research to generation

Every `/run` endpoint follows the same shape: validate prerequisites, reserve credits, queue a Celery job, return `202 { "job_id": "job_...", "stage": "research", "estimated_credits": 6 }`. Output is read back with GET on the same sub-resource once the job completes. Users can edit output at `needs_review` before the next stage runs - edits are PATCHes, never re-runs.

Endpoint	Request and response shape
POST `/projects/{id}/research/run` **GET** `/projects/{id}/research`	Body: `depth` (`standard|deep`), optional `focus_questions[]` (max 5). Result: `findings[]` - each with `claim`, `source_url`, `source_title`, `retrieved_at`. Findings without a source URL are rejected by the worker, not stored.
POST `/projects/{id}/script/run` **GET/PATCH** `/projects/{id}/script`	Body: `tone` (`explainer|story|list`), `hook_style`. Result: `sections[]` with `id`, `narration`, `on_screen_text`, `est_duration_sec`, `visual_intent`. PATCH edits one section by ID; word count and total estimate update in the response.
POST `/projects/{id}/voice/run`	Two modes. `{ "mode": "ai", "voice_id": "v_...", "section_ids": [...] }` generates per-section audio. `{ "mode": "recorded", "upload_id": "upl_..." }` attaches a user recording (Section 9). Result lands on `GET /voice`: per-section `asset_id`, `duration_sec`, `transcript_status`.
POST `/projects/{id}/voice/cleanup` **GET** `/projects/{id}/voice/captions-source`	Runs breath/filler removal and loudness normalisation (-14 LUFS) on recorded or AI voice, then faster-whisper transcription. Result: cleaned `asset_id` plus word-level timings `words[] { w, start, end }` - this timing file is the single source for captions and timeline sync.
POST `/projects/{id}/visual-plan/run` **GET/PATCH** `/projects/{id}/visual-plan`	Result: `shots[]` aligned to script sections - `shot_id`, `type` (`stock|generated_image|generated_video|screen|chart`), `query_or_prompt`, `duration_sec`. PATCH lets the user swap a shot type or rewrite a prompt before any credits are spent on generation.
POST `/projects/{id}/assets/generate` **GET** `/projects/{id}/assets`	Body: `shot_ids[]`. Stock shots resolve from licensed Pexels/Pixabay by default and cost 0 generation credits. Generated shots bill per asset; the response itemises the estimate before queuing. Each finished asset: `asset_id`, `kind`, `signed_url`, `license` (`source`, `license_name`, `attribution_required`).

> **YouTube source rule:** There is no endpoint that downloads YouTube clips by default. A workspace-level opt-in flag (`youtube_source_enabled`, owner-only, set via `PATCH /workspace/settings` with an explicit rights acknowledgement) unlocks `POST /projects/{id}/assets/import-url`. The spec ships the endpoint disabled; enabling it is a legal decision, not a technical one.

5. Post stages: motion, timeline, captions, render, publish

Endpoint	Request and response shape
POST `/projects/{id}/motion/run` **GET/PATCH** `/projects/{id}/motion`	Generates Remotion motion graphics from the visual plan: `elements[]` with `type` (`lower_third|chart|tier_list|stat_card|map_pin|callout`), `data` (values come from research findings, never free-typed by the model), `start_sec`, `duration_sec`, `style_tokens`. PATCH adjusts data and timing; style tokens come from the project theme, not per-element colours.
GET `/projects/{id}/timeline` **PATCH** `/projects/{id}/timeline`	The timeline is data, not a video file: `{ version, tracks[] { type: video|voice|music|caption|motion, clips[] { asset_id, in, out, start, props } } }`. PATCH sends `base_version` plus an operations list (`move`, `trim`, `split`, `replace_asset`, `set_props`). Stale `base_version` returns `409 VERSION_CONFLICT` with the current version - the client re-reads and replays. Every PATCH increments `version`; the last 20 versions are kept and restorable via `POST /timeline/restore { version }`.
POST `/projects/{id}/captions/run` **GET/PATCH** `/projects/{id}/captions`	Builds caption cues from the word-timing file (Section 4). Result: `cues[] { start, end, text }` plus style (`preset`, `max_chars_per_line: 42`). PATCH edits cue text; timings only change when the user drags a cue boundary, sent as explicit `start/end`.
POST `/projects/{id}/audio-mix/run`	Body: `music_asset_id` (licensed library only in MVP), `music_volume_db`, `ducking: true|false`, `sfx: [{ asset_id, at_sec }]`. Output is a mix settings object on the timeline, applied at render - no separate audio file is rendered until export.
POST `/projects/{id}/previews` **GET** `/projects/{id}/previews/{pid}`	Low-res Remotion preview render (480p, watermarked). Job-based. Result: `signed_url` (MP4) and `timeline_version` it was rendered from, so a stale preview is visibly stale in the UI.
POST `/projects/{id}/renders` **GET** `/projects/{id}/renders/{rid}`	Body: `resolution` (`1080p` in MVP; `4k` returns `422 NOT_AVAILABLE_IN_PLAN` until v2), `timeline_version`. Credits are estimated from duration and resolution and reserved before queuing. Result: `signed_url`, `duration_sec`, `size_bytes`, `credits_charged` (actual, settled on completion - see Section 8).
POST `/projects/{id}/publish-pack/run` **GET** `/projects/{id}/publish-pack`	Derives the distribution pack from a finished render: 9:16 Short/Reel cut (auto-reframed around the active speaker/subject), 1:1 cut, 3 thumbnail candidates (frames + title treatment), `title_options[]` (max 5), `description`, `hashtags[]`. Each item is a downloadable asset with its own `signed_url`. The API generates the pack - posting to YouTube or Instagram is done by the user in MVP; there is no publish-to-platform endpoint.

6. Jobs, polling, and webhooks

One job model covers research, voice, generation, preview, render, and publish pack. If a client can poll one job, it can poll all of them.

Endpoint	Notes
GET `/jobs/{job_id}`	`{ id, type, project_id, status, progress_pct, stage, error, result_ref, credits: { reserved, charged }, created_at, finished_at }`. `status`: `queued | running | succeeded | failed | cancelled`. `progress_pct` is honest or null - workers report real stage checkpoints; the API never fabricates a smooth progress bar.
GET `/projects/{id}/jobs`	Job history for a project, newest first. Used to rebuild the activity panel after a reload.
POST `/jobs/{job_id}/cancel`	Cancels if `queued` or early `running`. Reserved credits are released in full on cancel; a render past 80% cannot be cancelled and returns `409 JOB_NOT_CANCELLABLE`.
POST `/jobs/{job_id}/retry`	Re-queues a `failed` job with the same inputs and the same idempotency key lineage. Failed jobs are never auto-charged twice: the retry settles against the original reservation if it is still held.

Polling contract

- Poll `GET /jobs/{id}` every 3 seconds while the project page is open; back off to 10 seconds after 2 minutes. Stop on any terminal status.
- Job reads are cheap and sit in the highest rate-limit bucket, so polling never starves editing calls.
- Server-sent events are not in the MVP contract - polling plus webhooks cover both the browser and integrations. See the open decision in Section 10.

Webhooks

Endpoint / event	Notes
POST/GET/DELETE `/webhooks`, `/webhooks/{id}`	Register an HTTPS callback URL with an event list. Secret is generated at creation and shown once; it signs every delivery.
`job.succeeded` / `job.failed`	Payload: the full job object from `GET /jobs/{id}`, plus `event`, `event_id`, `sent_at`.
`render.completed`	Adds `render_id` and a 24-hour download `signed_url` so integrations can fetch without an extra call.
`credits.low`	Fires at 20% of the monthly allowance remaining. Once per threshold crossing, not per job.

Deliveries are signed with HMAC-SHA256 in `X-AVS-Signature` (timestamp + body). A delivery is retried 5 times over 24 hours with exponential backoff on any non-2xx response; after that the endpoint is paused and the owner is emailed. Receivers should dedupe on `event_id` - retries can duplicate a delivery that actually succeeded.

7. MCP tools

The same capabilities are exposed over the Model Context Protocol (MCP) so an assistant in a desktop client can drive a project by conversation. MCP tools call the same FastAPI services as REST - there is no second business logic path - and authenticate with a scoped API key (Section 1).

> **Proposal rule:** MCP edits never write directly. A tool that changes the script, timeline, or captions returns a *proposal* - a diff against the current version - which the user accepts or rejects in the web UI or by a follow-up tool call. This keeps conversational editing from silently destroying a timeline.

Tool	Input	Output and behaviour
`project_create`	`title, topic, target_length_sec, language`	Creates a project. Returns the project object. Direct write - creation destroys nothing.
`project_get` / `project_list`	`project_id` / filters	Read-only. Includes `stage_status` so the assistant can say what is done and what is next.
`stage_run`	`project_id, stage, params`	Queues the same job as the REST `/run` endpoint. Returns `job_id` and `estimated_credits`. Refuses with the REST error code when prerequisites or credits are missing.
`job_status`	`job_id`	Read-only poll, same shape as `GET /jobs/{id}`.
`script_propose_edit`	`project_id, section_id, new_narration`	Returns `proposal_id` with before/after text and the new duration estimate. Nothing changes yet.
`timeline_propose_ops`	`project_id, base_version, ops[]`	Same operations vocabulary as the timeline PATCH. Returns a proposal with the resulting duration and any clips it displaces.
`captions_propose_edit`	`project_id, cue_id, new_text`	Proposal only, like script edits.
`proposal_apply` / `proposal_reject`	`proposal_id`	Apply re-checks the base version: if the timeline moved since the proposal, apply fails with `VERSION_CONFLICT` and the proposal must be regenerated. Proposals expire after 24 hours.
`render_request`	`project_id, resolution, timeline_version`	Queues a render job exactly like REST. With `dry_run: true` it returns the credit estimate first; the assistant confirms cost with the user when the estimate exceeds 50 credits.
`credits_balance`	-	Read-only: balance, reserved, this-period usage. There is deliberately no MCP tool that buys credits.

MCP transport: streamable HTTP at `https://api.aivideostudio.app/mcp`, same versioning as REST (the v1 tool set is frozen with v1). Tool errors use the same `code` values as REST errors, so one error table serves both surfaces.

8. Credits and billing endpoints

Pricing is credits on a subscription because every render and generation carries a real compute cost. All third-party costs behind credit pricing are estimates and ranges - see the Pricing doc; no cost figure is stated as fact in this spec.

Endpoint	Notes
GET `/billing/balance`	`{ balance, reserved, period_allowance, period_used, resets_at }`.
GET `/billing/ledger`	Every credit movement: `reserve`, `settle`, `release`, `grant`, `purchase` - with `job_id` where relevant. This ledger is the source of truth in any billing dispute.
GET `/billing/estimate`	Query: `?action=render&duration_sec=&resolution=` or `?action=generate&kind=&count=`. Returns the estimate the UI shows before the user confirms. Estimates are honoured: a job never settles above its estimate.
POST `/billing/topup`	Owner only. Creates a payment session with the payment provider and returns its checkout URL; credits land via the provider webhook, never via this response.

Reserve-settle flow: a job reserves its estimate at queue time, settles the actual (never above the estimate) on success, and releases the full reservation on failure or cancel. The user is therefore never charged for a failed render - that rule is enforced in the API, not promised in marketing copy.

9. Uploads and storage

Endpoint	Notes
POST `/uploads`	Body: `filename`, `content_type`, `size_bytes`, `purpose` (`voice_recording|music|image|broll`). Returns `upload_id` and a signed PUT URL for direct upload to R2/S3. The API never receives the file bytes.
POST `/uploads/{id}/complete`	Client calls after the PUT succeeds. The server verifies size and type, probes the file with ffprobe, and creates the asset. A mismatched type returns `400 VALIDATION_ERROR` and the object is deleted.
GET `/assets/{id}`	Metadata plus a fresh `signed_url`. Limits: voice recording max 60 min / 500 MB; images max 25 MB; b-roll max 2 GB (multipart signed URLs issued automatically above 100 MB).
DELETE `/assets/{id}`	Removes the asset from the project and marks the object for deletion in 24 hours. Assets referenced by a timeline clip cannot be deleted - `409 ASSET_IN_USE` with the referencing clip IDs.

Storage layout is an implementation detail behind these endpoints; clients only ever hold asset IDs and short-lived signed URLs. Nothing in the API exposes a bucket name or a permanent public URL.

10. Open decision and build order

> **Open decision - realtime preview transport.** MVP polls jobs and renders previews as files (decided above). The open question is only about live preview while editing: **Option A:** stay with file previews in v1, add a streaming Remotion preview in v2. **Option B:** invest in a WebSocket preview session in v1, adding an estimated 1-2 weeks (estimate) to the MVP. Recommendation: Option A - it keeps the 6-8 week MVP intact, and the polling contract already covers progress. Decide before the spike ends; everything else in this spec is decided.

Build order this spec implies

- **Spike (5-7 days):** auth, project create/get, one `/run` stage with the full job + polling loop, one signed-URL upload. Proves the contract end to end.
- **MVP (6-8 weeks):** all stage endpoints, timeline operations with versioning, credits reserve-settle, webhooks, MCP read + propose tools.
- **v1 (+4-6 weeks):** publish pack refinements, proposal apply flows in the UI, top-up billing, higher rate-limit tiers.

Endpoints marked 4K, publish-to-platform, and YouTube import are intentionally absent or gated above. Adding any of them is a spec change with a new version of this document - not a silent extension of v1.